Tarea de Bases de Datos SQL

Creación de una Base de Datos: ArteVida Cultural

Carlos San Román Cazorla

Universidad Complutense de Madrid

Máster de IA, Big Data y Data Science

Febrero 2026

v. 1.0.0

Índice general

1. Enunciado **3**
1.1. Objetivo del proyecto 3
1.2. Enunciado 3
1.3. Diseño conceptual 3
1.4. Diseño lógico 4
1.5. Implementación 4
1.5.1. Consideraciones 4
1.6. Documentación a entregar 5

2. Diseño conceptual **6**
2.1. Definición de entidades 6
2.2. Definición de relaciones 8

3. Paso a tablas **12**

4. Implementación. Script Completo SQL **14**

1 Enunciado

1.1. Objetivo del proyecto

Análisis, diseño e implementación de una base de datos.

1.2. Enunciado

Nuestra empresa, conocida como "ArteVida Cultural", se dedica a la organización de eventos culturales únicos, que abarcan desde conciertos de música clásica hasta exposiciones de arte vanguardista, pasando por obras de teatro y conferencias. Es crucial gestionar la diversidad de eventos y detalles, así como las ganancias obtenidas. Para ello, es necesario llevar un registro adecuado de cada evento, de las actividades que se organizan, de los artistas que los protagonizan, las ubicaciones, la venta de entradas y el entusiasmo de los asistentes, quienes valorarán cada evento con un número del 0 al 5, sólo si han asistido al evento.

En nuestra empresa ofrecemos actividades con un nombre y un tipo: conciertos de música (clásica, pop, blues, soul, rock and roll, jazz, reggaeton, góspel, country, etc.), exposiciones, obras de teatro y conferencias. En cada actividad participa uno o varios artistas y tiene un coste (suma del caché de los artistas). El artista tiene un nombre, no tiene un caché fijo, el caché depende de la actividad en la que participe y una breve biografía. Un artista puede participar en muchas actividades.

La ubicación tendrá un nombre (Teatro María Guerrero, Estadio Santiago Bernabeu, etc.), dirección, ciudad o pueblo, aforo, precio del alquiler y características. De cada evento necesitamos saber el nombre del evento (p.e. "VI festival de música clásica de Alcobendas"), la ubicación, el precio de la entrada, la fecha y la hora, así como una breve descripción. En un evento solo se realiza una actividad, aunque la actividad se puede realizar en varios eventos.

También tendremos en cuenta los asistentes a los eventos, de los que sabemos su nombre completo, teléfonos de contacto y email. Una persona puede asistir a más de un evento y a un evento pueden asistir varias personas, controlando que no se exceda el aforo de la ubicación en la que se va a realizar el evento.

Nos interesará realizar consultas de eventos por tipo de actividad, número de eventos en los que se ha realizado cada actividad, fecha con más eventos, ciudad en la que se han realizado más eventos teniendo en cuenta que en varias ciudades se pueden haber realizado el mismo número de evento, actividades con un solo artista, ciudad con solo eventos de teatro, evento con más ceros en su valoración, etc.

1.3. Diseño conceptual

A partir de los requisitos del apartado anterior se construirá el modelo conceptual de datos, en concreto el modelo entidad-relación. Para ello, puedes utilizar herramientas online que te permiten realizar los diagramas muy similares a los de los apuntes de clase.

Se requiere describir **brevemente** el porqué de la existencia de las entidades (claves, tipos de los atributos: multivalorados, derivados, ..., dominios, ...), y de las relaciones expresando su cardinalidad mínima y máxima. Además, enumerar los conceptos que no están expresados en la modelización porque el modelo entidad-relación no lo permite y necesitamos tenerlo en cuenta para los siguientes pasos del diseño. A veces en esta fase en necesario replantearse la fase anterior, pueden surgir detalles no escritos en la fase de especificación, aprovecha para aportar tus matices al enunciado.

1.4. Diseño lógico

En esta tercera fase del proyecto se aplican las técnicas aprendidas en el curso en el apartado "**paso a tablas**", para transformar el diagrama entidad-relación obtenido en la fase 2 en su correspondiente modelo relacional, es decir, en un conjunto de relaciones con las claves primarias y ajenas, las relaciones se representan en líneas con el nombre de la relación y los atributos entre paréntesis separados por comas.

1.5. Implementación

La implementación en MySQL deberá estar bien presentada, puesto que el código que vamos a escribir en la creación **refleja lo visto en las fases anteriores** y es el resultado final del trabajo. Se pide coherencia en el proceso. Es importante la organización y presentación porque facilita el mantenimiento posterior. Se recomienda utilizar identificadores significativos, comentarios, separando la definición de las tablas, la inserción de datos y las consultas. Pero **todo en un solo script**.

En la definición de la estructura se utilizarán los tipos de datos apropiados a cada columna, se utilizarán restricciones sobre los atributos, restricciones mediante reglas y restricciones sobre los dominios, se definirán las claves y las relaciones entre las tablas.

Los datos se pueden insertar de forma variada según se ha indicado en el documento **"Cargar datos en workbench"**. Si tenéis problemas con la inserción de datos con archivos, podéis insertar los datos con insert into. En cualquiera de los casos han de ser suficientes para poder comprobar que las operaciones son correctas.

Las consultas que se realizarán serán **variadas** y con **distinto nivel de complejidad** (unas 10 consultas). Se pide calidad en la consulta. También se realizará alguna vista para ser utilizada alguna consulta, y al menos un trigger para controlar el buen funcionamiento de la base de datos.

NO está permitido generar el código de creación de la base de datos con MySQL Workbench, el código generado de esta forma no deja ver con claridad el proceso, introduce mucho código en principio innecesario. Escribir las instrucciones con los comandos vistos en clase de SQL.

1.5.1. Consideraciones

Para aprobar el proyecto es imprescindible que el script entregado sea compilable y su ejecución correcta. Es fundamental **la coherencia en todo el proceso** de creación de la base de datos desde el modelo entidad-relación hasta la implementación. Se valorará la complejidad y variedad de las consultas.

Aunque sabemos que lo aprendido en este máster es para utilizarlo con una gran cantidad de datos, nos interesa que introduzcáis los datos necesarios para realizar las consultas y que así podáis comprobar que están bien. No os podemos pasar los conjuntos de datos porque estaríamos obligando a hacer un diseño del problema, y esto es parte de la tarea, diseñar la base de datos.

1.6. Documentación a entregar

- **Un solo documento PDF cuyo nombre será el nombre y apellido(s) del autor**, por ejemplo, si el documento fuera mío se llamaría IsabelRiomoros.pdf. El documento debe contener:
  - Portada con el nombre del autor
  - Diseño e-r
  - Modelo relacional
  - Script completo copiado
- **Un sólo script SQL** que contenga el código de construcción de la base de datos, las instrucciones de carga de los datos y las consultas. El nombre del script será el **nombre del autor** sin espacios en blanco.

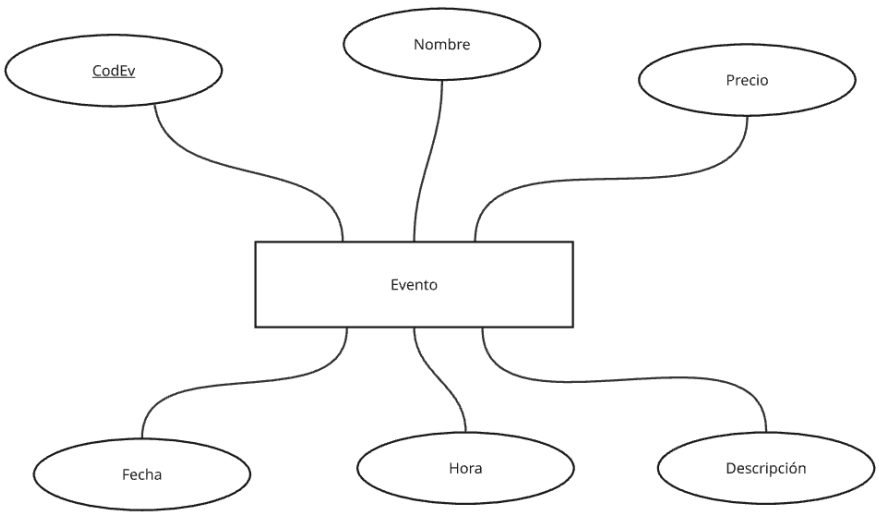
Si fuera necesario, se adjuntarán los ficheros de datos externos que se utilizan para cargar los datos de las tablas. En principio, no necesito ejecutar los scripts para comprobar que funcionan. En este caso es mejor calidad de los datos que cantidad.

2 Diseño conceptual

Comenzaremos definiendo el modelo Entidad-Relación que se pide para diseñar la base de datos del ejercicio propuesto. Para ello, iremos definiendo entidad a entidad con los atributos que se piden en el enunciado y posteriormente, pasaremos a definir las relaciones mediante las cuales interaccionan.

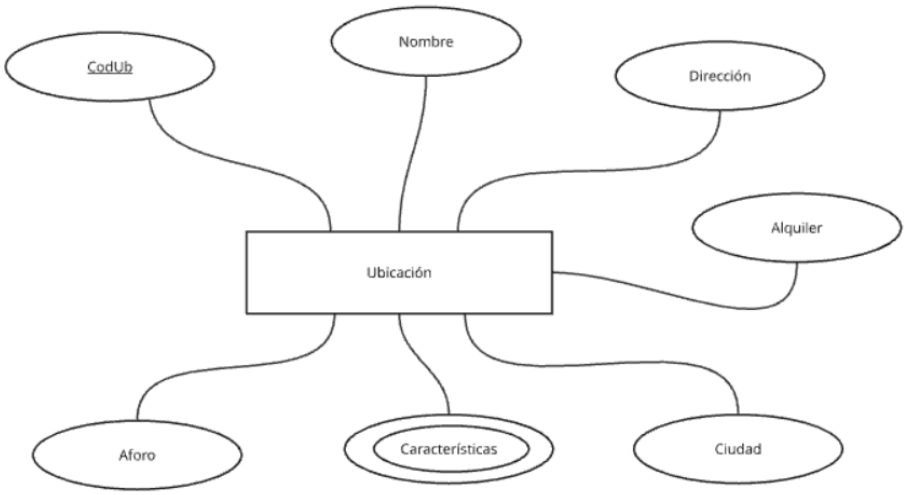
2.1. Definición de entidades

1. **Evento:** Está claro que evento es una entidad al ser un objeto abstracto que ocurre en el mundo real y es algo distinguible de todos los demás objetos. En el enunciado, se pide que cada evento debe estar caracterizado por un nombre, el precio, la fecha, la hora y una breve descripción de lo que tratará el evento. Para tener un código identificador del evento, que nos sirva de *primary key*, añadiremos un atributo CodEv completando así los atributos (Puntualizamos que el Nombre no valdría como primary key ya que, por ejemplo, dos obras de teatro pueden tener el mismo nombre si se realiza la misma obra pero son dos eventos distintos). Cabe destacar que todos los atributos son simples ya que no se descomponen en otros más simples o no pueden contener más de un valor simultáneamente. Por tanto, el esquema entidad-relación de la entidad Evento quedará definido de la siguiente manera:

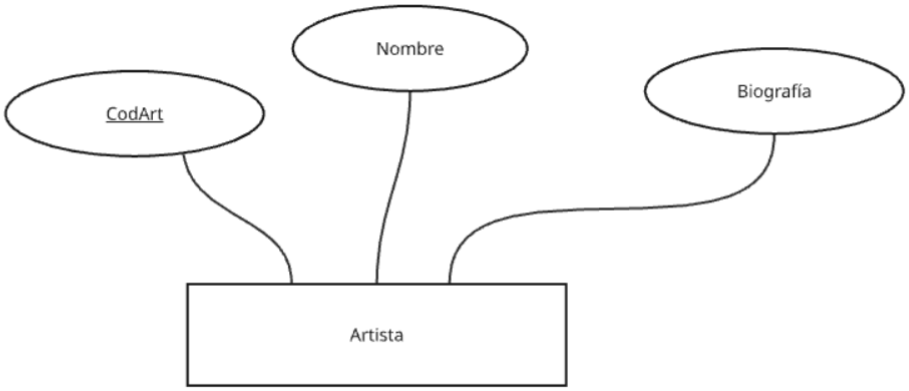


2. **Ubicación:** Una ubicación tiene identidad propia dentro del sistema y existe independientemente de los eventos que se realicen en ella. Esto encaja perfectamente con una entidad fuerte. Además, a partir del enunciado, se infieren las siguientes características: Nombre, Dirección, Ciudad, Aforo, Alquiler y Características. Además, al igual que en la entidad Evento, definiremos un atributo CodEv

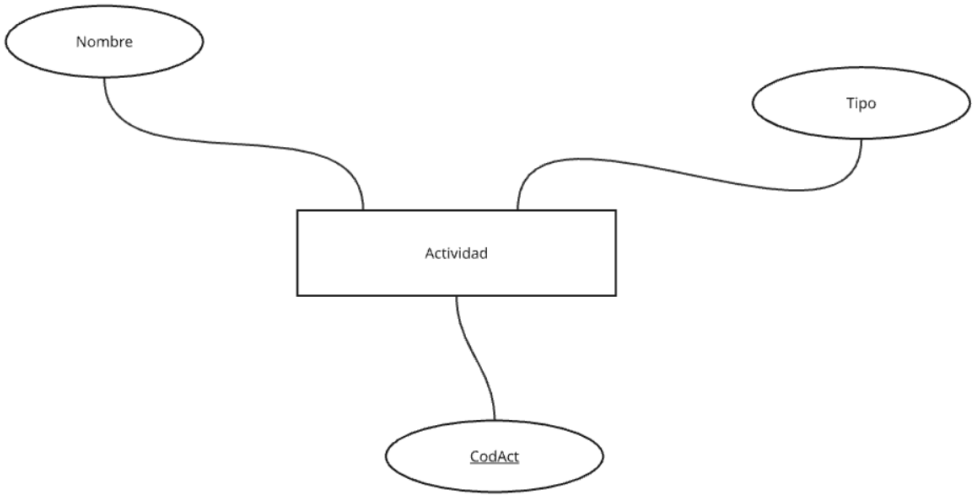
que nos sirva como *primary key* ya que el resto de atributos se podrían repetir. También, volvemos a resaltar que los atributos en esta entidad son simples por las mismas razones que anteriormente, excepto *Características*, el cual es multivalorado al poder albergar múltiples adjetivos que describan la ubicación, es decir, una ubicación puede tener varias características.



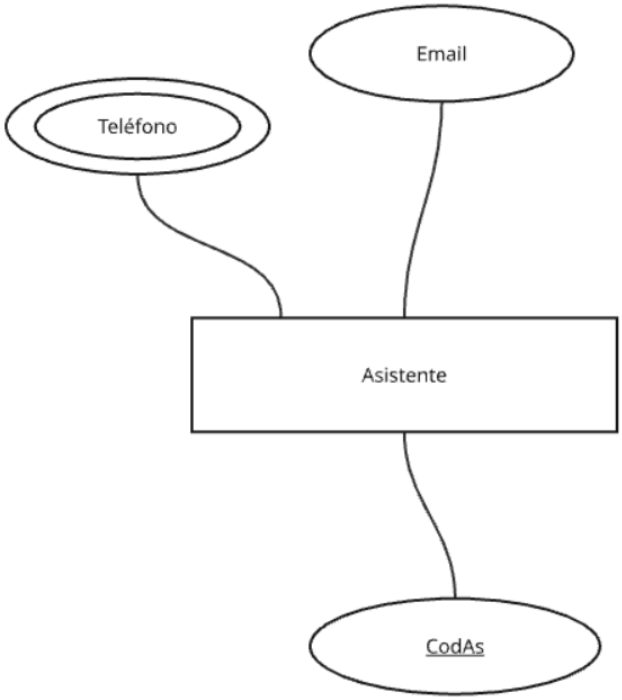
3. **Artista:** Un artista tiene existencia propia dentro del sistema y sus datos no dependen de una actividad concreta, lo que la convierte en una entidad fuerte. A partir del enunciado, se deducen las siguientes características: Nombre y Biografía. Definiremos un atributo CodArt que nos sirva como *primary key* ya que el resto de atributos se podrían repetir. También, volvemos a resaltar que los atributos en esta entidad son simples ya que el nombre no se pide separarlo en nombre y apellidos (por lo que no es necesario considerarlo como compuesto) y biografía es una descripción del artista como texto único.



4. **Actividad:** Una actividad tiene identidad propia dentro del sistema y muchas de las consultas que se harán giran en torno a las actividades, lo que refuerza su papel como entidad. Como atributos, podemos destacar el Nombre y el Tipo con un atributo CodAct que nos sirva como *primary key*. En este caso, todos los atributos son simples.



5. **Asistente:** Una asistente representa a una persona real, con identidad propia que puede asistir a varios eventos y sus datos no dependen de de un evento concreto. Como atributos, podemos destacar el Nombre, el Teléfono y el email junto con un atributo CodAs que nos sirva como *primary key*. En este caso, los atributos son simples, excepto el teléfono ya que un asistente puede tener varios teléfonos simultáneamente.

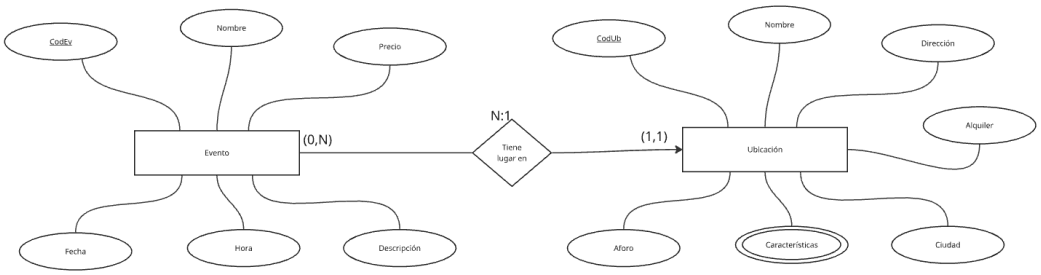


2.2. Definición de relaciones

1. **Tiene lugar en:** Se trata de la asociación entre evento y ubicación. Para estudiar las cardinalidades no hacemos las siguientes preguntas:

- Un evento, ¿en cuántas ubicaciones puede tener lugar como mínimo y como máximo? A lo sumo, puede estar ocurriendo en un único lugar ya que todos los participantes y asistentes no pueden estar en dos sitios distintos a la vez. Por otro lado, si el evento existe, tiene que estar ocurriendo en algún sitio, por lo que la cardinalidad es (1,1).
- Una ubicación, ¿cuántos eventos puede albergar como mínimo y como máximo? Una ubicación puede que no contrate ningún evento, mientras que también podría darse que contratase múltiples eventos. Por tanto, la cardinalidad es (0,N).

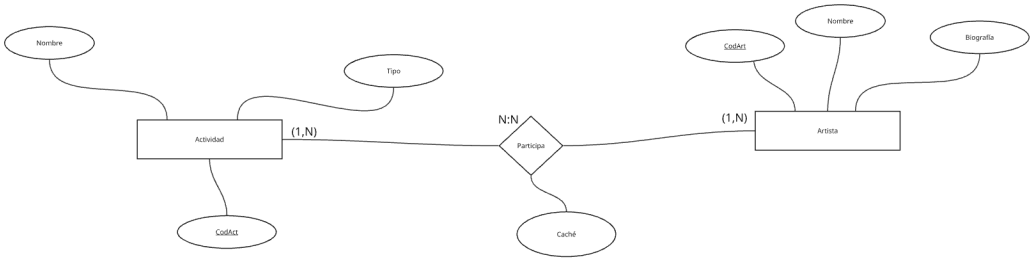
El esquema entidad-relación que modela esta relación quedaría de la siguiente manera:



2. **Participa:** Asociación entre actividad y artista. Para estudiar las cardinalidades no hacemos las siguientes preguntas:

   - Un artista, ¿en cuántas actividades puede participar como mínimo y como máximo? Por el enunciado, un artista puede realizar múltiples actividades. Por otro lado, si el artista existe, es decir, es un ente presente debe realizar alguna actividad. Por tanto, la cardinalidad es (1,N).
   - En una actividad, ¿cuántos artistas pueden participar como mínimo y como máximo? Por el enunciado, obtenemos que en cada actividad participa uno o varios artistas. Luego, su cardinalidad es (1,N).

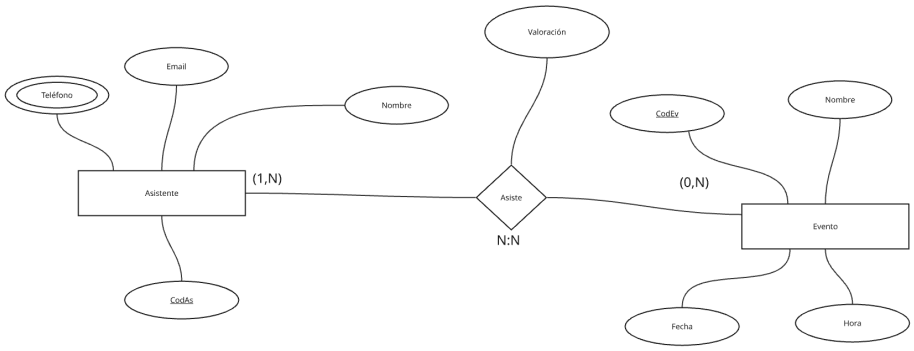
   Además, en función de la actividad, el caché del artista varía, luego el caché es un atributo de la relación. De ahí, se puede derivar el coste de la actividad como la suma de los cachés de los artistas, luego obtenemos un atributo derivado que es el Coste (No lo pondremos como parte del modelo ya que resultaría redundante y al hacer el modelo relacional estaríamos almacenando una columna innecesaria perdiendo eficiencia en memoria). El esquema entidad-relación que modela esta relación quedaría de la siguiente manera:



3. **Asiste:** Asociación entre evento y asistente. Para estudiar las cardinalidades no hacemos las siguientes preguntas:

   - Un asistente, ¿a cuántos eventos puede asistir como mínimo y como máximo? Por el enunciado, una persona puede asistir a más de un evento. Por otro lado, puede que la persona no asista a ningún evento. Por tanto, la cardinalidad es (0,N).
   - A un evento, ¿cuántos asistentes asisten como mínimo y como máximo? Si el evento existe, acudirá al menos una persona, ya que en caso contrario, no se celebraría y no existiría el evento. Por otro lado, en el enunciado, se expone que a un evento asisten varias personas siempre y cuando no se exceda el aforo. Luego, su cardinalidad es (1,N).

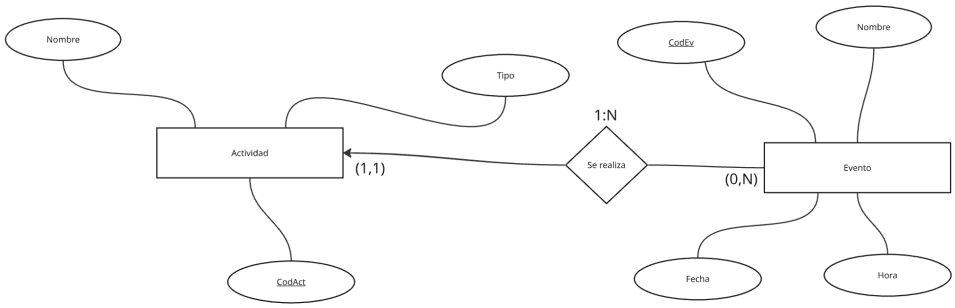
   Además, se quiere llevar una medición de las valoraciones de los usuarios del evento. Luego, por cada usuario que asista, existirá una valoración asociada a la relación, la cual trataremos como atributo. **Observación:** La restricción del control de aforo no puede ser representada en el diagrama de entidad-relación, por lo que modelizaremos eso al construir la base de datos mediante un trigger. Por tanto, el esquema entidad-relación que modela esta relación quedaría de la siguiente manera:



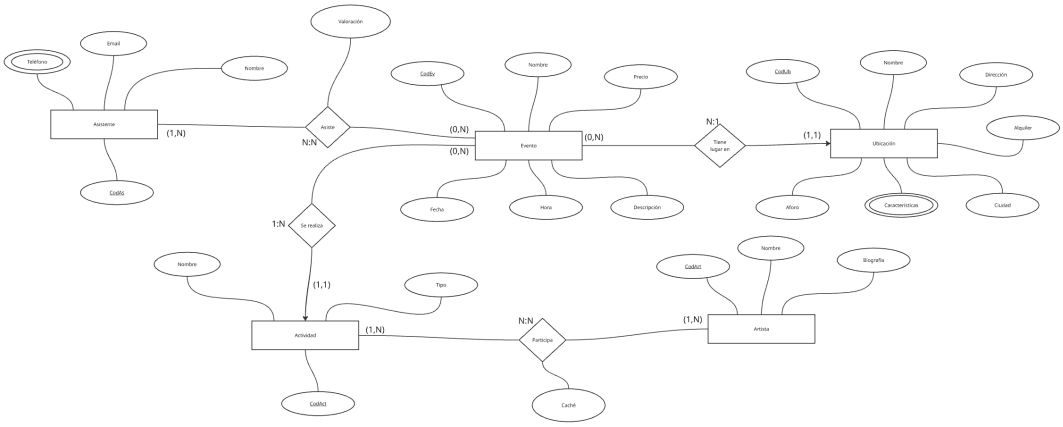
4. **Se realiza:** Asociación entre evento y actividad. Para estudiar las cardinalidades no hacemos las siguientes preguntas:

   - Una actividad, ¿en cuántos eventos se realiza como mínimo y como máximo? Por el enunciado, una actividad se puede realizar en múltiples eventos. Por otro lado, puede que esa actividad no se realice en ninguno. Por tanto, la cardinalidad es (0,N).
   - En un evento, ¿cuántas actividades se realizan como mínimo y como máximo? Por el enunciado, en cada evento solo se realiza una actividad. Luego, su cardinalidad es (1,1).

   Por tanto, el esquema entidad-relación que modela esta relación quedaría de la siguiente manera:



Por tanto, el modelo E-R completo quedará de la siguiente manera:



3 Paso a tablas

Construyamos las tablas, recordando las reglas que debemos de seguir para realizar este paso. Tengamos en cuenta que hemos tenemos casos particulares de elementos multivalorados con los que actuaremos de forma específica. También, destacar que no hemos considerado atributos compuestos (en los Nombres), por los que los trataremos como simples.

1. **Toda entidad fuerte se representa como tabla:** Sea E un conjunto de entidades fuertes con atributos descriptivos $a_1, a_2, ..., a_n$, se representa la entidad mediante una tabla con n columnas, cada una de ellas correspondiente a un atributo.

2. **Una relación se puede considerar como una tabla de valores:**

   a) En una **relación varios a varios**, se representa un esquema con atributos para las claves primarias de los dos conjuntos de entidades involucradas y cualquier atributo descriptivo del conjunto de relaciones.

   b) En una **relación varios a uno o uno a varios**, totales en el lado de varios, se pueden representar añadiendo un atributo extra en el lado de varios, que contenga la clave primaria del uno.

   c) Para **relaciones uno a uno**, se puede elegir cualquiera de los lados para actuar como lado "varios". Es decir, los atributos se pueden añadir a cualquiera de las tablas correspondientes a los dos conjuntos de entidades.

   d) Si la **participación es parcial en el lado varios**, la sustitución de una tabla por un atributo extra en la relación correspondiente al lado "varios"podría tener como resultado valores nulos.

3. **Atributos multivalorados:** Para los atributos multivalorados se crean nuevas tablas. Para el atributo multivalorado M, se crea una tabla T con una columna C que corresponde a la clave primaria del conjunto de entidades o conjuntos de relaciones del que M es atributo. Cada valor del atributo M se representa como una única fila en la tabla.

Así, con estas reglas, particularizando a nuestro ejercicio, obtenemos las siguientes tablas:

- ubicación(<u>CodUb</u>, Nombre, Dirección, Alquiler, Ciudad, Aforo): El aforo se controlará con un trigger.

- características(<u>CodUb</u>, <u>Característica</u>): Para cada ubicación se solicita guardar sus características, luego es un atributo multivalorado. Por ello, se necesita una tabla para el propio atributo. Como una ubicación puede tener varias características, consideraremos la clave primaria como el conjunto de los atributos que forman la tabla. A su vez, <u>CodUb</u> es una clave foránea de la tabla *ubicación*.

- artista(<u>CodArt</u>, Nombre, Biografía)

- actividad(CodAct, Nombre, Tipo): Para completar la actividad, vemos que el tipo Concierto tiene una gran cantidad de subtipos correspondientes con los géneros musicales. Luego construiremos dos tablas adicionales: una tabla *genero* para definir los tipos de música y una tabla *actividad_genero* que servirá de puente entre ambas tablas.
- genero(CodGenero, NombreGenero)
- actividadGenero(CodAct,CodGenero): La clave primaria es el conjunto de los dos atributos para conseguir la unicidad y ambos atributos son claves foráneas.
- asistente(CodAs, Nombre, Email): En este caso, se podría haber interpretado el nombre como un atributo compuesto y separarlo en simples como Nombre y Apellidos. Debido a que no es necesario para el ejercicio, decidimos tratar el nombre completo como un atributo simple.
- telefono(CodAs, Teléfono): Para cada asistente se solicita guardar sus teléfonos de contactos, luego es un atributo multivalorado. Por ello, se necesita una tabla para el propio atributo. Como una persona puede tener varios números, consideraremos la clave primaria como el conjunto de los atributos que forman la tabla. A su vez, CodAs es una clave foránea de la tabla *asistente*.
- participa(CodArt, CodAct, Caché): Como el caché del artista depende de la actividad en la que participe, el coste total de la actividad dependerá de los cachés de los artistas que participen. Así, mediante la suma de estos cachés podríamos obtener el coste total con una simple consulta, por lo que no será necesario guardar una columna en la tabla con la que se obtendría información repetida.
- asiste(CodAs, CodEv, Valoración)
- evento(CodEv, Nombre, Precio, Descripción, Hora, Fecha, *CodUb*, *CodAct*): Como Evento se relaciona con la entidad Actividad mediante *Se realiza* a través de una relación $1 : N$ parcial se añade como atributo *CodAct*. Lo mismo ocurre con *CodUb*, convirtiéndose ambas dos en claves foráneas.

4 Implementación. Script Completo SQL

Para realizar la implementación de la base de datos en SQL a través de las tablas que hemos obtenido en el paso anterior, utilizamos mySQL. Tal y como se indica en el enunciado, tenemos en cuenta que la cantidad de datos se ha limitado para poder comprobar la corrección de las consultas y el buen funcionamiento de la base de datos. Para la inserción de datos no se emplean bases de datos auxiliares, simplemente se añadirán datos a mano con **insert into**.

Notación: Durante el desarrollo del código SQL, para tratar de diferenciar entre la construcción del modelo y las consultas, se ha seguido la siguiente notación:

- En los primeros cuatro puntos del código (construcción de tablas, triggers y vistas, e inserción de datos) los comandos SQL se escriben en mayúsculas para resaltar su significado teórico. Los tipos de datos se escriben en minúscula ya que es algo particular de cada atributo del problema.
- En el punto 5, las consultas, como son particulares del problema, se han escrito en minúscula para mostrar que no tienen un significado teórico dentro del modelo, simplemente consecuencias de la base de datos construida.

A continuación se muestra el código SQL de implementación de la base de este problema:

```
DROP DATABASE IF EXISTS ArteVidaCultural;
CREATE DATABASE ArteVidaCultural;
USE ArteVidaCultural;

/*
=============================================
||   1. CREACIÓN DE TABLAS  DE ENTIDADES   ||
=============================================
*/
CREATE TABLE ubicacion (
   CodUb varchar(4) primary key,
   Nombre varchar(40) not null,
   Direccion varchar(40) not null,
   Ciudad varchar(9) not null,
   Alquiler numeric(6,2) not null,
   Aforo int not null
);

CREATE TABLE caracteristicas (
   CodUb  varchar(4),
   Caracteristica   varchar(40) not null,
   PRIMARY KEY(CodUb, Caracteristica),
   FOREIGN KEY(CodUb) REFERENCES ubicacion(CodUb) ON DELETE CASCADE ON UPDATE CASCADE
 );

  CREATE TABLE artista (
```

```sql
   CodArt varchar(4) primary key,
   Nombre varchar(40) not null,
   Biografia text
);

 CREATE TABLE actividad (
   CodAct varchar(4) primary key,
   Nombre varchar(40) not null,
   Tipo varchar(40) not null
);

 CREATE TABLE genero (
   CodGenero varchar(4) primary key,
   NombreGenero varchar(40) not null
);

 CREATE TABLE actividadGenero (
   CodAct varchar(4),
   CodGenero varchar(4) not null,
   PRIMARY KEY(CodAct, CodGenero),
   FOREIGN KEY(CodAct) REFERENCES actividad(CodAct) ON DELETE CASCADE
                                                     ON UPDATE CASCADE,
   FOREIGN KEY(CodGenero) REFERENCES genero(CodGenero) ON DELETE CASCADE
                                                       ON UPDATE CASCADE
);

 CREATE TABLE asistente (
   CodAs varchar(4) primary key,
   Nombre varchar(40) not null,
   Email varchar(40) not null
);

 CREATE TABLE telefono (
   CodAs varchar(4),
   -- Ponemos numTelefono para distinguir ente tabla y atributo
   numTelefono varchar(9) not null check(numTelefono REGEXP '^[0-9]{9}$'),
   PRIMARY KEY(CodAs, numTelefono),
   FOREIGN KEY(CodAs) REFERENCES asistente(CodAs) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE evento (
   CodEv varchar(4) primary key,
   Nombre varchar(40) not null,
   Precio numeric(6,2) not null,
   Descripcion TEXT,
   Fecha date not null,
   Hora time not null,
   CodUb varchar(50),
   CodAct varchar(50),
   -- Añadiremos una restricción para que no haya dos eventos
   -- en el mismo sitio a la misma hora
   CONSTRAINT unicoEventoFechaHoraUbi UNIQUE (Fecha, Hora, CodUb),
   -- Marcamos las claves ajenas
   FOREIGN KEY(CodAct) REFERENCES actividad(CodAct) ON DELETE CASCADE
                                                     ON UPDATE CASCADE,
   FOREIGN KEY(CodUb) REFERENCES ubicacion(CodUb) ON DELETE CASCADE ON UPDATE CASCADE
);

/*
```

```
=============================================
||   2. CREACIÓN DE TABLAS  DE RELACIONES  ||
=============================================
*/
 CREATE TABLE participa (
   CodArt varchar(4),
   CodAct varchar(4),
   -- A diferencia del documento teórico ponemos nCache ya que Cache
   -- es una palabra reservada del programa
   nCache numeric(9,2) not null,
   PRIMARY KEY(CodArt, CodAct),
   FOREIGN KEY(CodArt) REFERENCES artista(CodArt) ON DELETE CASCADE
                                                  ON UPDATE CASCADE,
   FOREIGN KEY(CodAct) REFERENCES actividad(CodAct) ON DELETE CASCADE
                                                    ON UPDATE CASCADE
);
 CREATE TABLE asiste (
   CodAs varchar(4),
   CodEv varchar(4),
   Valoracion tinyint not null check(Valoracion between 0 and 5),
   PRIMARY KEY(CodAs, CodEv),
   FOREIGN KEY(CodAs) REFERENCES asistente(CodAs) ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY(CodEv) REFERENCES evento(CodEv) ON DELETE CASCADE ON UPDATE CASCADE
);

/*
=============================================
||          3. TRIGGERS Y VISTAS           ||
=============================================
*/

-- Comenzamos viendo el trigger para el control de aforo

DELIMITER //

CREATE TRIGGER controlAforo
BEFORE INSERT ON asiste
FOR EACH ROW
BEGIN
    DECLARE aforo_max INT;
    DECLARE asistentes_actuales INT;

    -- 1. Obtenemos el aforo máximo permitido para el evento
    SELECT u.Aforo INTO aforo_max
    FROM evento e INNER JOIN ubicacion u ON e.CodUb = u.CodUb
    WHERE e.CodEv = NEW.CodEv;

    -- 2. Contamos cuántos asistentes hay ya registrados en ese evento
    SELECT COUNT(*) INTO asistentes_actuales
    FROM asiste
    WHERE CodEv = NEW.CodEv;

    -- 3. Comparamos y lanzamos error si está lleno
    IF asistentes_actuales >= aforo_max THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El aforo para este evento ya está completo.';
    END IF;
END //
```

```sql
DELIMITER ;

-- Creamos otro trigger para controlar que un artista no esté
-- en dos eventos el mismo día
DELIMITER //

CREATE TRIGGER tg_artista_disponibilidad_fecha
BEFORE INSERT ON participa
FOR EACH ROW
BEGIN
    DECLARE numEventos INT;
    SELECT COUNT(e.CodEv) INTO numEventos
    FROM evento e INNER JOIN participa p ON e.CodAct = p.CodAct
    WHERE p.CodArt = NEW.CodArt AND  e.Fecha IN ( SELECT Fecha
                                                  FROM evento
                                                  WHERE  CodAct = NEW.CodAct);
    -- Si numEventos >= 1 ,  el artista ya participa en un evento en esa fecha
    IF numEventos >= 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Este artista ya tiene programado otro evento
        en la misma fecha.';
        END IF;
END //
DELIMITER ;

-- Creamos una vista que muestra cuánta gente se ha inscrito en cada evento
-- en comparación con el aforo total y cuánto dinero se ha recaudado

CREATE VIEW vista_ocupacion_eventos AS
SELECT
    e.Nombre AS Evento,
    e.Fecha,
    u.Nombre AS Lugar,
    u.Aforo AS Capacidad_Max,
    COUNT(asi.CodAs) AS Asistentes_Inscritos,
    (u.Aforo - COUNT(asi.CodAs)) AS Plazas_Libres,
    (COUNT(asi.CodAs) * e.Precio) AS Recaudacion_Total
FROM evento e
INNER JOIN ubicacion u ON e.CodUb = u.CodUb
LEFT JOIN asiste asi ON e.CodEv = asi.CodEv
GROUP BY e.CodEv;

-- Creamos una vista que permita ver de un solo vistazo qué artista
-- tiene que ir a qué lugar, en qué fecha y qué actividad realizará.
CREATE VIEW vista_agenda_artistas AS
SELECT
    a.Nombre AS Artista,
    act.Nombre AS Actividad,
    e.Fecha,
    e.Hora,
    u.Nombre AS Ubicacion,
    u.Ciudad,
    p.nCache AS Pago_Acordado
FROM artista a
INNER JOIN participa p ON a.CodArt = p.CodArt
INNER JOIN actividad act ON p.CodAct = act.CodAct
INNER JOIN evento e ON act.CodAct = e.CodAct
INNER JOIN ubicacion u ON e.CodUb = u.CodUb;
/*
```

```sql
=============================================
||        4. INSERCIÓN DE DATA SET          ||
=============================================
*/
-- Utilizamos la ayuda de la IA para conseguir datos correspondientes al problema

-- 1. UBICACIONES
INSERT INTO ubicacion VALUES
('U001', 'Teatro Real Madrid', 'Plaza de Isabel II', 'Madrid', 850.00, 10),
('U002', 'Palau de la Música', 'C/ Palau de la Música', 'Barcelona', 600.00, 50),
('U003', 'Centro Cultural Las Cigarreras', 'Calle San Carlos',
'Alicante', 150.00, 100);

-- 2. CARACTERÍSTICAS
INSERT INTO caracteristicas VALUES
('U001', 'Acústica Premium'), ('U001', 'Escenario Giratorio'),
('U002', 'Patrimonio Humanidad'), ('U003', 'Espacio Abierto');

-- 3. ARTISTAS
INSERT INTO artista VALUES
('AR01', 'Elena Ferrante', 'Pianista clásica de renombre internacional.'),
('AR02', 'Marc Sanchis', 'Especialista en teatro de calle y performance.'),
('AR03', 'Sara Baras', 'Bailaora de flamenco contemporáneo.'),
('AR04', 'Dúo Armonía', 'Dúo de violonchelo y violín.'),
('AR05', 'Roberto Carlos', 'Pintor expresionista y muralista.');

-- 4. ACTIVIDADES
INSERT INTO actividad VALUES
('AC01', 'Recital de Piano', 'Música Clásica'),
('AC02', 'Noche de Flamenco', 'Danza'),
('AC03', 'Taller de Expresión Corporal', 'Teatro'),
('AC04', 'Exposición Muralismo', 'Artes Plásticas'),
('AC05', 'Concierto de Cuerda', 'Música');

-- 5. GÉNEROS
INSERT INTO genero VALUES
('G001', 'Clásico'), ('G002', 'Folklore'), ('G003', 'Experimental'),
('G004', 'Moderno');

-- 6. ACTIVIDAD-GÉNERO
INSERT INTO actividadGenero VALUES
('AC01', 'G001'), ('AC02', 'G002'), ('AC03', 'G003'), ('AC04', 'G004'),
('AC05', 'G001');

-- 7. PARTICIPA (Artistas en Actividades)
INSERT INTO participa VALUES
('AR01', 'AC01', 1500.00),
('AR03', 'AC02', 2000.00),
('AR02', 'AC03', 800.00),
('AR05', 'AC04', 500.00),
('AR04', 'AC05', 1200.00);

-- 8. EVENTOS
INSERT INTO evento VALUES
('E001', 'Gala de Invierno', 45.00, 'Evento inaugural', '2024-12-01',
'20:00:00', 'U001', 'AC01'),
('E002', 'Pasión Flamenca', 30.00, 'Espectáculo de danza', '2024-12-05',
'21:30:00', 'U002', 'AC02'),
('E003', 'Cuerdas en el Palau', 25.00, 'Concierto íntimo', '2024-12-10',
```

```sql
'19:00:00', 'U002', 'AC05'),
('E004', 'Murales Vivos', 5.00, 'Exposición guiada', '2024-12-15',
'10:00:00', 'U003', 'AC04'),
('E005', 'Mover el Alma', 15.00, 'Workshop intensivo', '2024-12-20',
'17:00:00', 'U003', 'AC03'),
('E006', 'Piano bajo las estrellas', 50.00, 'Recital nocturno', '2024-12-22',
'22:00:00', 'U001', 'AC01'),
('E007', 'Navidad Clásica', 35.00, 'Concierto festivo', '2024-12-25',
'18:00:00', 'U001', 'AC05'),
('E008', 'Arte en la Calle', 0.00, 'Evento gratuito', '2025-01-05',
'12:00:00', 'U003', 'AC04'),
('E009', 'Flamenco y Olé', 30.00, 'Segunda función', '2025-01-10',
'21:30:00', 'U002', 'AC02'),
('E010', 'Despedida Elena', 60.00, 'Último concierto', '2025-01-15',
'20:00:00', 'U001', 'AC01'),
('E011', 'Los Miserables', 80.00, 'Última actuación', '2027-01-15',
'20:00:00', 'U001', 'AC01');

-- 9. ASISTENTES
INSERT INTO asistente VALUES
('A001', 'Carlos Ruiz', 'carlos@mail.com'), ('A002', 'Ana Belén', 'ana@mail.com'),
('A003', 'Luis Miguel', 'luis@mail.com'), ('A004', 'Marta Sánchez', 'marta@mail.com'),
('A005', 'Jorge Sanz', 'jorge@mail.com'), ('A006', 'Lucía Gil', 'lucia@mail.com'),
('A007', 'Pedro Marín', 'pedro@mail.com'), ('A008', 'Elena Poe', 'elena@mail.com'),
('A009', 'Raúl Giner', 'raul@mail.com'), ('A010', 'Sofía Loren', 'sofia@mail.com'),
('A011', 'Marcos Paz', 'marcos@mail.com'), ('A012', 'Julia Otero', 'julia@mail.com'),
('A013', 'Iván Ferreiro', 'ivan@mail.com'), ('A014', 'Nerea Ruiz', 'nerea@mail.com'),
('A015', 'Óscar Casas', 'oscar@mail.com'), ('A016', 'Paula Eche', 'paula@mail.com'),
('A017', 'Quique Gon', 'quique@mail.com'), ('A018', 'Rocío Jurado', 'rocio@mail.com'),
('A019', 'Sergio Dalma', 'sergio@mail.com'),
('A020', 'Tania Llasera', 'tania@mail.com');

-- 10. TELÉFONOS
INSERT INTO telefono VALUES
('A001', '600111222'), ('A002', '611222333'), ('A003', '622333444'),
('A004', '633444555'), ('A005', '600111222'), ('A006', '655666777'),
('A007', '666777888'), ('A008', '677888999'), ('A009', '688999000'),
('A010', '699000111'), ('A011', '601000111'), ('A012', '602000222'),
('A013', '603000333'), ('A014', '604000444'), ('A015', '605000555'),
('A016', '606000666'), ('A017', '607000777'), ('A018', '608000888'),
('A019', '609000999'), ('A020', '610111222');

-- 11. ASISTENCIA (Llenamos algunos eventos)
-- Vamos a llenar el evento E001 (Aforo 10 en U001) para probar el trigger
INSERT INTO asiste VALUES
('A001', 'E001', 5), ('A002', 'E001', 4), ('A003', 'E001', 5), ('A004', 'E001', 3),
('A005', 'E001', 5), ('A006', 'E001', 4), ('A007', 'E001', 2), ('A008', 'E001', 5),
('A009', 'E001', 4), ('A010', 'E001', 5);

-- Otros eventos con menos gente
INSERT INTO asiste VALUES
('A011', 'E002', 5), ('A012', 'E002', 4), ('A013', 'E003', 3), ('A014', 'E004', 5),
('A015', 'E005', 4), ('A016', 'E006', 5), ('A017', 'E007', 4), ('A018', 'E008', 3);

/*
=============================================
||              5. CONSULTAS               ||
=============================================
*/
```

```sql
-- 1. Comenzamos probando las vistas para ver su correcto funcionamiento
select * from vista_agenda_artistas;
select * from vista_ocupacion_eventos;

-- 2.  Probamos el funcionamiento del trigger de aforo: Para ello,
-- intentamos añadir un nuevo asistente a un evento completo (Nos da error)
INSERT INTO asiste VALUES ('A011', 'E001', 5);

-- A continuación, probamos el trigger que comprueba que un artista no puede tener
-- dos eventos el mismo día. Para ello, tomamos al artista Elena Ferrante (AR01),
-- con actividad AC01 (Recital de Piano) vinculada a tres eventos.
-- Creamos una actividad nueva
INSERT INTO actividad VALUES ('AC06', 'Charla Magistral', 'Educativa');

-- Creamos un evento para esa actividad el mismo día que el Evento E001 (2024-12-01)
INSERT INTO evento VALUES
('E011', 'Masterclass Piano', 10.00, 'Charla técnica', '2024-12-01',
'10:00:00', 'U003', 'AC06');

-- Esto falla:
INSERT INTO participa VALUES ('AR01', 'AC06', 300.00);

-- 3. Finalmente, empezamos las consultas solicitadas en el ejercicio:

-- Número de eventos en los que se realiza cada actividad
select e.codAct, ac.Nombre as 'Nombre de la actividad' , count(e.codAct)
as 'Número de eventos'
from evento e inner join actividad ac on e.CodAct = ac.CodAct
group by e.CodAct ;

-- Listado de las calificaciones de los eventos según las valoraciones
-- de los asistentes
select e.CodEv, e.Nombre as 'Nombre del evento', count(asi.CodAs)
as 'Número de asistentes', u.Nombre as 'Lugar', round(avg(asi.Valoracion),2)
as 'Valoracion Media',
case
    when avg(asi.Valoracion) >= 4.5 then 'Excelente'
    when avg(asi.Valoracion) >= 3 then 'Buena'
    when avg(asi.Valoracion) >= 2 then 'Regular'
    else 'Mala'
end as 'Clasificación'
from evento e inner join asiste asi on e.CodEv = asi.CodEv
              inner join ubicacion u on u.CodUb = e.CodUb
group by e.CodEv
order by avg(asi.Valoracion) desc, count(asi.CodAs) desc ;

-- Número de personas que acuden a cada tipo de actividad
select a.Tipo, count(asiste.CodAs) as 'Número total de asistentes a la actividad'
from evento e inner join actividad a on e.CodAct = a.CodAct
              inner join asiste on asiste.CodEv = e.CodEv
group by a.Tipo;

-- Top 3 artistas más rentables
select a.Nombre as Artista, count(e.CodEv) as 'Número Eventos',
       sum(p.nCache) as 'Coste Total Caché',
       sum(e.Precio * sub.Total_Asistentes) as 'Recaudación Generada',
       (sum(e.Precio * sub.Total_Asistentes) - sum(p.nCache)) as 'Beneficio Neto'
from artista a inner join participa p on a.CodArt = p.CodArt
```

```sql
                inner join evento e on p.CodAct = e.CodAct
inner join (
    -- Subconsulta para contar asistentes por evento
    select CodEv, count(*) AS 'Total Asistentes'
    from asiste
    group by CodEv
) sub on e.CodEv = sub.CodEv
group by a.CodArt
order by Beneficio_Neto desc
-- Si queremos ver el balance de los otros artistas
-- basta con eliminar la siguiente condición
limit 3;

-- Ubicaciones con peor funcionamiento: Consideraremos un mal funcionamiento cuando
-- tengamos una ocupación con menos del 50%
select u.Nombre as Ubicacion, u.Ciudad, u.Aforo as 'Capacidad Max',
    round(avg(asistencias.Total), 2) as 'Promedio Asistentes',
    round((avg(asistencias.Total) / u.Aforo) * 100, 2) as 'Porcentaje Ocupacion Media'
from ubicacion u inner join evento e on u.CodUb = e.CodUb
inner join (
    -- Contamos asistentes por cada evento en esa ubicación
    select CodEv, count(CodAs) as 'Total'
    from asiste
    group by CodEv
) asistencias on e.CodEv = asistencias.CodEv
group by u.CodUb
having 'Porcentaje Ocupacion Media' < 50
order by 'Porcentaje Ocupacion Media';

-- Consulta el último evento visitado de cada usuario
select a.Nombre, max(e.Fecha) as UltimoEvento
from asistente a inner join asiste asi on a.CodAs = asi.CodAs
                 inner join evento e on asi.CodEv = e.CodEv
group by a.Nombre
order by a.Nombre;


-- Consulta acerca de los usuarios que más han asistido a eventos,
-- la valoración media de cada usuario y el último evento visitado
select asi.Nombre, asi.Email, count(ast.CodEv) as Eventos_Visitados,
    round(avg(ast.Valoracion), 1) as Valoracion_Media_Asistente,
    (--  Hacemos una subconsulta para conectar el nombre del asistente
     -- con el último evento al que ha acudido
        select e.Nombre
        from asiste a2 inner join evento e on a2.CodEv = e.CodEv
        where a2.CodAs = asi.CodAs
        order by e.Fecha desc, e.Hora desc
        limit 1
    ) as Ultimo_Evento_Visto
from asistente asi inner join asiste ast on asi.CodAs = ast.CodAs
group by asi.CodAs, asi.Nombre, asi.Email
having Eventos_Visitados >= 1
order by Eventos_Visitados DESC;

-- Consulta para ver qué personas comparten teléfono
select asistente.CodAs, asistente.Nombre, t.numTelefono
from asistente inner join telefono t on asistente.CodAs = t.CodAs
where t.numTelefono in (select numTelefono from telefono
                        group by numTelefono having count(numTelefono) > 1);
```

```sql
-- Tiempo que falta para cada evento no celebrado
select e.CodEv, e.Nombre, e.Fecha,
    abs(timestampdiff(day, e.Fecha, curdate())) 'Días Restantes'
from evento e
where e.Fecha >= curdate()
order by e.Fecha;

-- Buscador de ubicaciones disponibles para undía solicitado:
-- En  este caso, tomaremos como ejemplo '2024-12-01'
select u.Nombre as 'Ubicación Disponible', u.Ciudad, u.Aforo
from ubicacion u
where u.CodUb not in (
    -- Subconsulta que busca ubicaciones ocupadas en una fecha
    select CodUb
    from evento
    where Fecha = '2024-12-01'
);

-- Mapa de Popularidad por Género
select g.NombreGenero, count(ast.CodAs) as 'Total Asistentes',
    round(avg(ast.Valoracion), 2) as 'Valoracion Media'
from genero g inner join actividadGenero ag on g.CodGenero = ag.CodGenero
              inner join evento e on ag.CodAct = e.CodAct
              left join asiste ast on e.CodEv = ast.CodEv
group by g.CodGenero
order by 'Total Asistentes' DESC;

-- Utilizamos la vista para consultar el total recaudado por cada evento
select Evento, Recaudacion_Total as 'Recaudación Total por evento'
from vista_ocupacion_eventos;

-- Por último, queremos saber cuál ha sido la recaudación total de todos los eventos
-- organizados por la empresa
select sum(Recaudacion_Total) as 'Recaudación Total'
from vista_ocupacion_eventos;
```